



WARWICK

Advanced Computer Graphics

Real Time Methods and Extensions

Dr. Tom Bashford-Rogers

Overview

- ▶ Sampling Improvements
 - Better Sampling
 - Better use of existing samples
 - Specialized Sampling
- ▶ Real time approximations
- ▶ Extensions



Path Guiding

- ▶ Recall Monte Carlo estimate of the Rendering Equation

$$L_o(x, \omega_o) = \frac{1}{N} \sum_{j=1}^N \frac{f_r(x, \omega_{i,j}, \omega_o) L_i(x, \omega_{i,j}) \cos(\theta_j)}{p_{\Omega}(\omega_{i,j})}$$

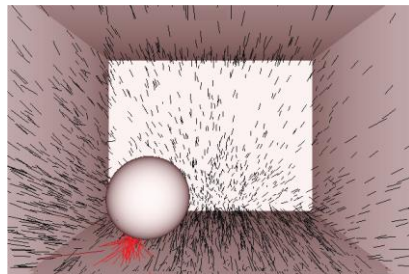
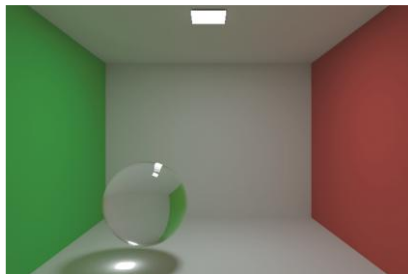
- ▶ Variance reduced is we can sample proportional to terms in the integrand



Path Guiding

- ▶ What if we can sample proportional to the incoming radiance

$$p_{\Omega}(\omega_i) \propto L_i(x, \omega_i)$$



- ▶ But this is what we want to estimate!

Path Guiding

- ▶ Idea: Cache estimates of incoming lighting in the scene
 - Use this to guide sampling of incident radiance
$$p_{\Omega}(\omega_i) \approx L_i(x, \omega_i)$$
 - Cache is an imperfect guess, but better than nothing
 - Often progressively refine cache

Path Guiding

- ▶ Only sampling incident radiance ignores BSDF
 - Can combine into product importance sampling $p_{\Omega}(\omega_i) \approx f_r(x, \omega_i, \omega_o) L_i(x, \omega_i) \cos(\theta)$
 - Often via projecting into the same sampleable basis
 - Or combine with MIS

Path Guiding

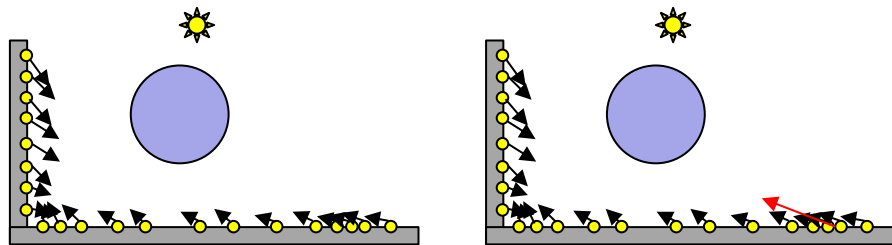
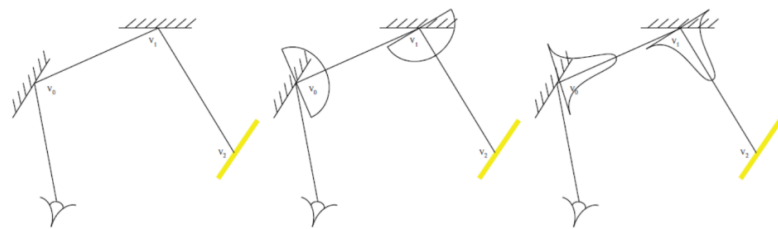
► Combine with MIS

$$L_o(x, \omega_o) = \frac{1}{N} \sum_{j=1}^N \frac{f_r(x_j, \omega_{i,j}, \omega_o) L_i(x, \omega_{i,j}) \cos(\theta_j)}{c_1 p_{f_r}(\omega_{i,j}) + c_2 p_{L_i}(\omega_{i,j})}$$

- Where c_1 and c_2 are probabilities of sampling BSDF $p_{f_r}(\omega_{i,j})$ or incident radiance $p_{L_i}(\omega_{i,j})$

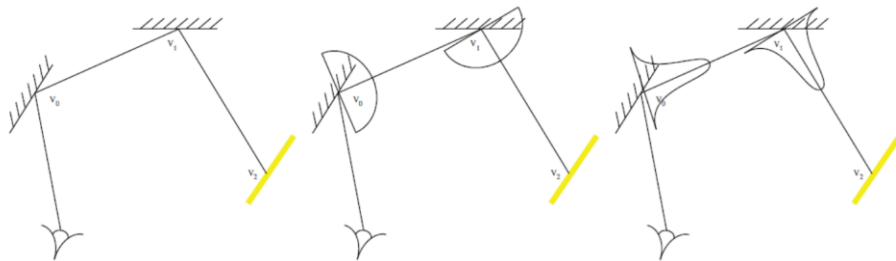
Path Guiding

- ▶ Can learn from
 - Tracing whole paths
 - Tracing photons



Path Guiding

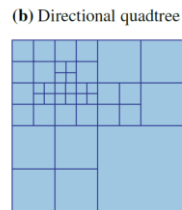
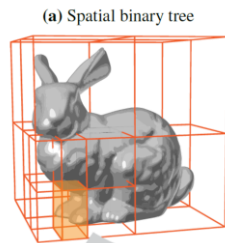
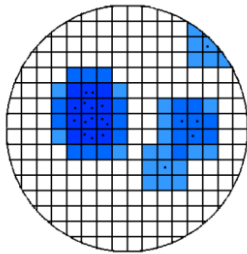
- ▶ Adapt distribution to incident radiance across scene
- ▶ Multiple Methods
 - Grids / Hierarchies
 - Basis Functions
 - Neural Networks



Path Guiding

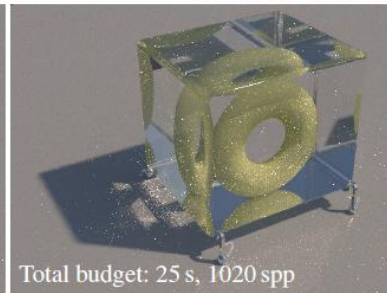
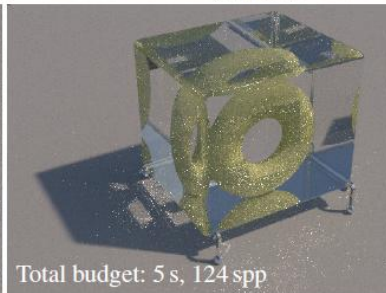
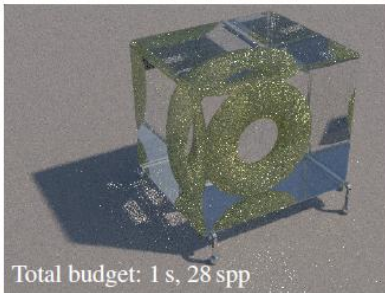
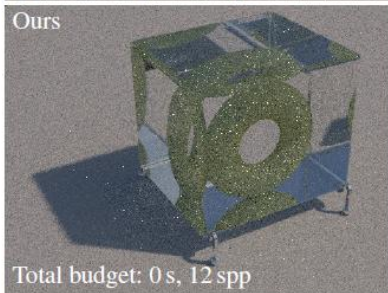
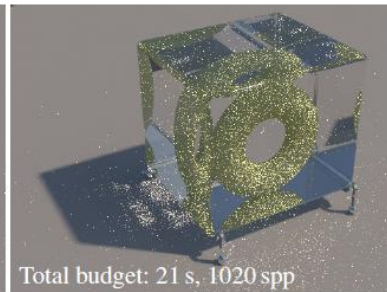
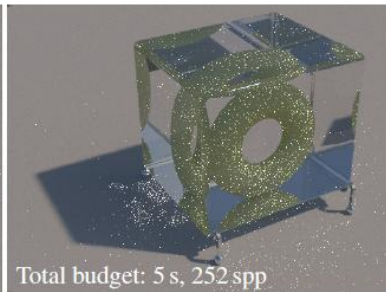
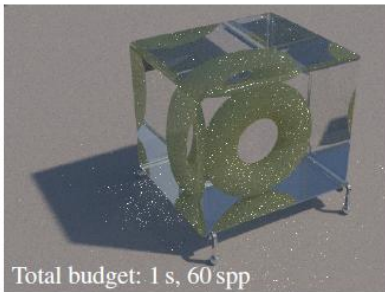
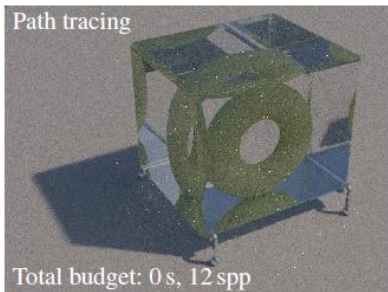
► Grids / Hierarchies

- Map (hemi)sphere to grid / hierarchy
- Project sampled incoming radiance into grid
- Sample grid / hierarchy for $p_{L_i}(\omega_{i,j})$



Path Guiding

► Grids / Hierarchies



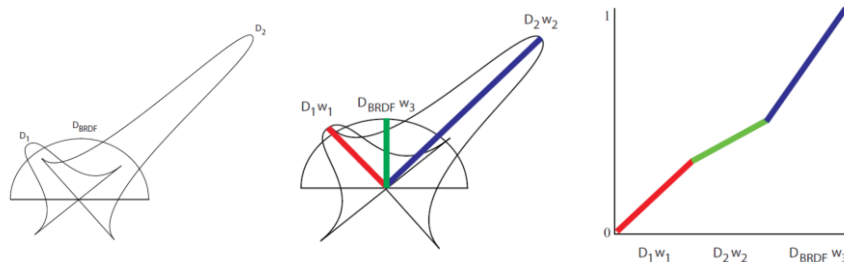
Path Guiding

► Basis Functions

– Integrate sampleable basis over incoming radiance

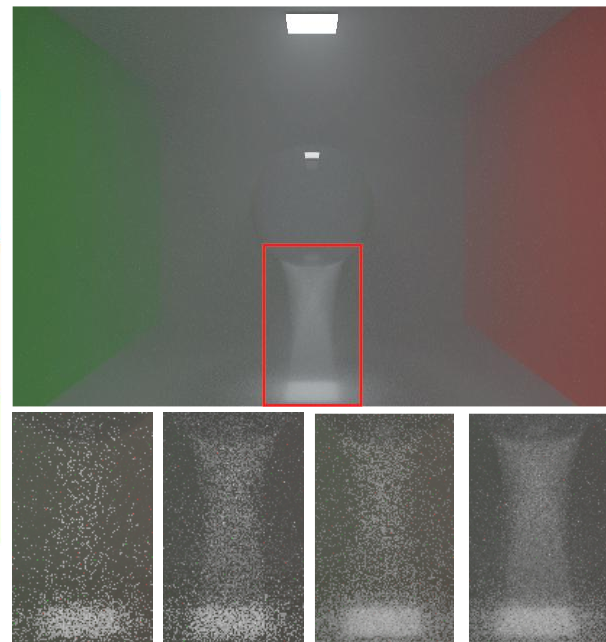
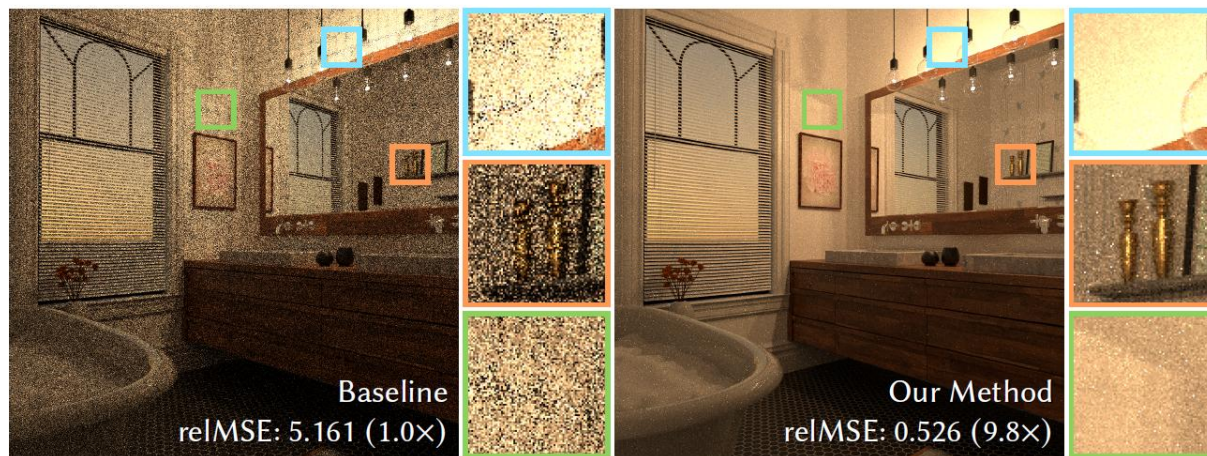
- Cosine Lobes (Me)
- Gaussians (Vorba)
- Von Mises-Fisher (Herlotz)

– Sample basis when generating samples



Path Guiding

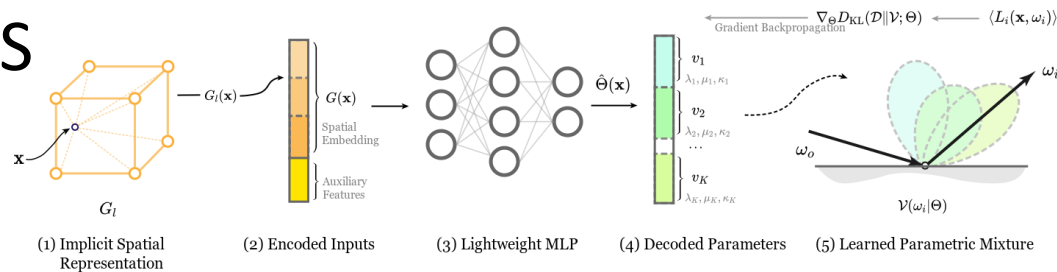
► Basis Functions



Path Guiding

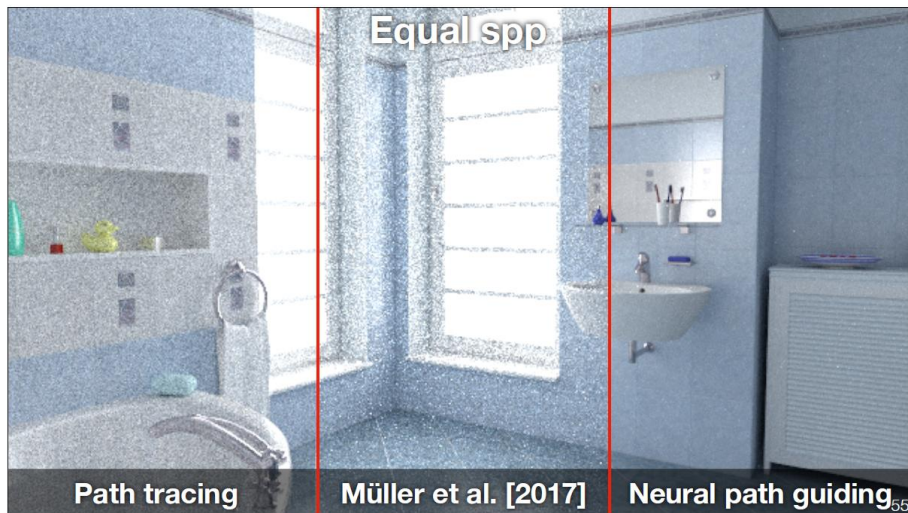
► Neural Networks

- Encode incident radiance in a Neural Network
 - Can also learn product of BSDF + incident radiance
- Learn to predict basis functions / grid / mapping from PSS



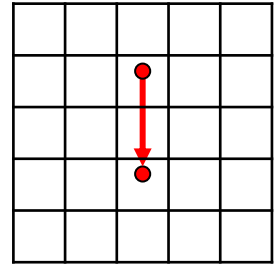
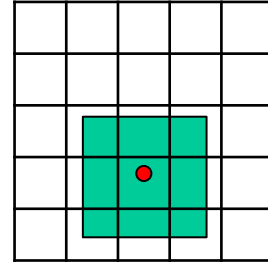
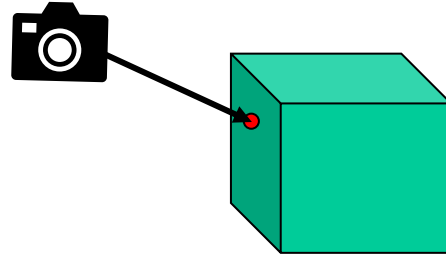
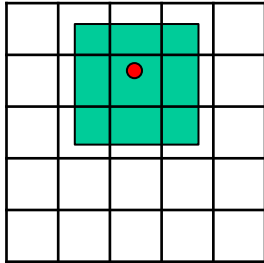
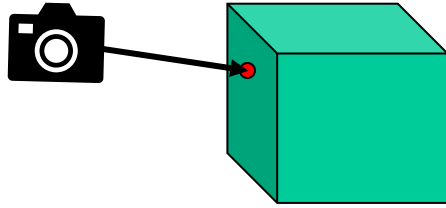
Path Guiding

► Neural Networks



Motion Vectors

- ▶ Describe per pixel movement between frames



Motion Vectors

- ▶ Compute via reprojection
 - Given a pixel in frame at time t
 - We want to know where it was at frame $t-1$
 - Unproject current pixel to world in frame t
 - Reproject to image plane at $t-1$



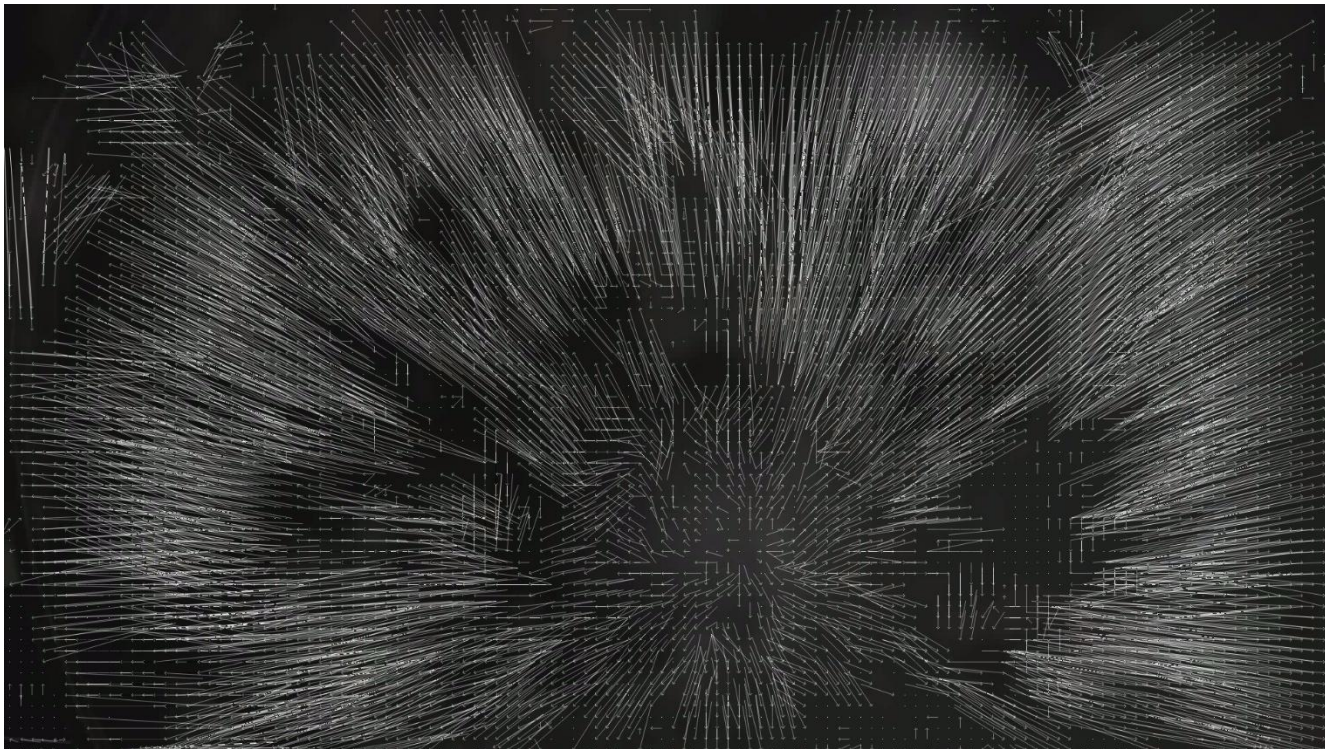
Motion Vectors

- ▶ Compute via reprojection
- ▶ Need
 - Projection Matrix P
 - V_t view matrix of current frame
 - V_{t-1} view matrix of previous frame

Motion Vectors

- ▶ Compute via reprojection
 - Create clip space coordinate
 - $x_{film,t} \in [0..W - 1]$
 $y_{film,t} \in [0..H - 1]$
 - $$p_{clip,t} = (\frac{2x_{film,t}}{W - 1} - 1, y_c = \frac{2y_{film,t}}{H - 1} - 1, z, 1)$$
 - Project to world
 - $p_{world,t} = \mathbf{V}_t^{-1} \mathbf{P}^{-1} p_{clip,t}$
 - Include perspective division
 - Project back to screen
 - $p_{clip,t-1} = \mathbf{P} \mathbf{V}_{t-1} p_{world,t}$
 - $$x_{film,t-1} = \frac{x_{clip,t-1} + 1}{2} W, \quad y_{film,t-1} = \frac{y_{clip,t-1} + 1}{2} H$$

Motion Vectors



Motion Vectors

- ▶ Can then use for various applications
 - ReSTIR
 - Motion Blur
 - Temporal Antialiasing



ReSTIR

- ▶ Can make better use of existing samples
 - Spatially
 - Temporally
- ▶ Reservoir-based Spatiotemporal Importance Resampling



ReSTIR

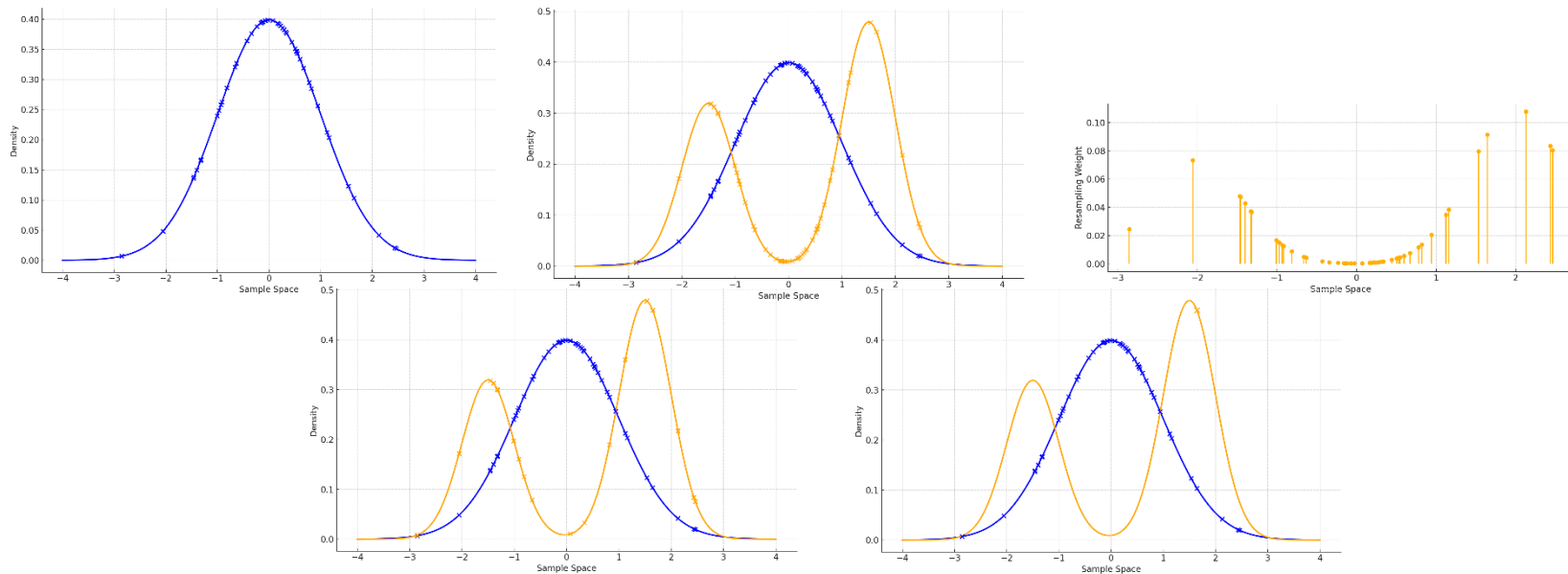
- ▶ Resampled Importance Sampling (RIS)
 - Given a set of samples $X_s = \{x_0, x_1, x_2, \dots, x_n\}$
 - Generated from PDFs $\{p(x_0), p(x_1), p(x_2), \dots, p(x_n)\}$
 - Want to choose one sample from X_s
distributed proportional to some other
distribution $g(x)$
 - Samples close to proportional to the product

ReSTIR

- ▶ Resampled Importance Sampling (RIS)
 - Compute unnormalized weight for each sample $w'_i = \frac{g(x_i)}{p(x_i)}$
 - Normalize $w_i = \frac{w'_i}{\sum_{j=0}^N w'_j}$
 - (Re)sample from X_s proportional to w

ReSTIR

► Resampled Importance Sampling (RIS)



ReSTIR

► Resampled Importance Sampling (RIS)

- Can be generalized
- Rewrite rendering equation as

$$L_o(x, \omega_o) = \frac{1}{N} \sum_{j=1}^N f_r(x, \omega_{i,j}, \omega_o) L_i(x, \omega_{i,j}) \cos(\theta_j) \cdot w_\mu(x_j)$$

- Where $w_\mu(x_j)$ is an *unbiased contribution weight*
- So far $w_\mu(x_j) = \frac{1}{p_\Omega(\omega_{i,j})}$

ReSTIR

► Resampled Importance Sampling (RIS)

- When resampling

$$w_{\mu}(x_j) = \frac{1}{g(\omega_{i,j})} \cdot \frac{1}{M} \sum_{k=1}^M \frac{g(\omega_{i,k})}{p(\omega_{i,k})}$$

- However can be an estimate

ReSTIR

► Reservoir Sampling

- Given a large (possibly infinite) pool of samples
 - Each has a weight
 - From RIS
- Unbiasedly select one

```
class Reservoir
    y ← 0 // The output sample
    wsum ← 0 // The sum of weights
    M ← 0 // The number of samples seen so far
    function update(xi, wi)
        wsum ← wsum + wi
        M ← M + 1
        if rand() < (wi/wsum) then
            y ← xi

    function reservoirSampling(S)
        Reservoir r
        for i ← 1 to M do
            r.update(S[i], weight(S[i]))
        return r
```


ReSTIR

► Reservoir Sampling

- Can combine samples taken at current point / pixel
- Can map other samples to point / pixel
- Can map samples from previous frame
 - Motion vectors
- Can combine reservoirs



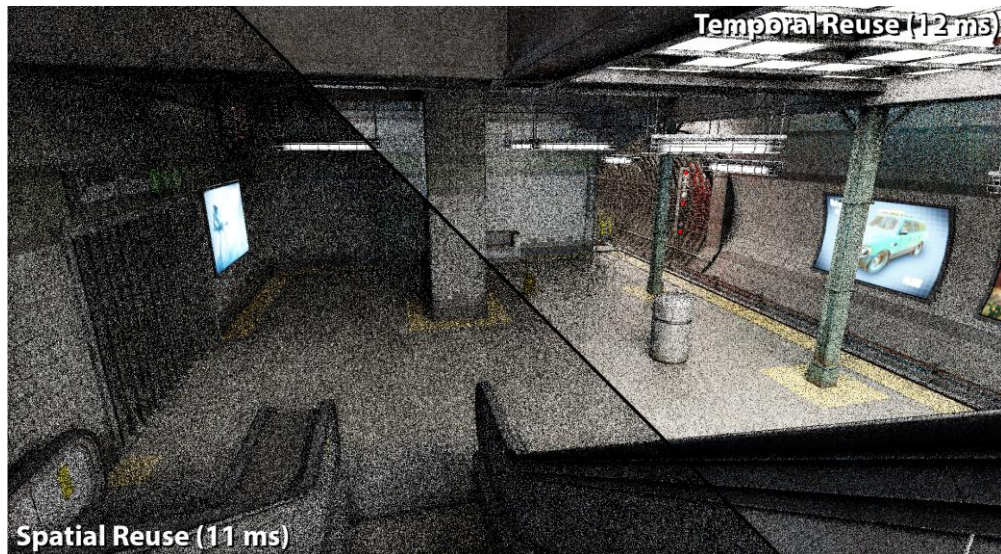
ReSTIR

- ▶ Combine Importance Resampling and Reservoir Sampling
- ▶ Can be used for Direct Lighting
- ▶ And Path Tracing



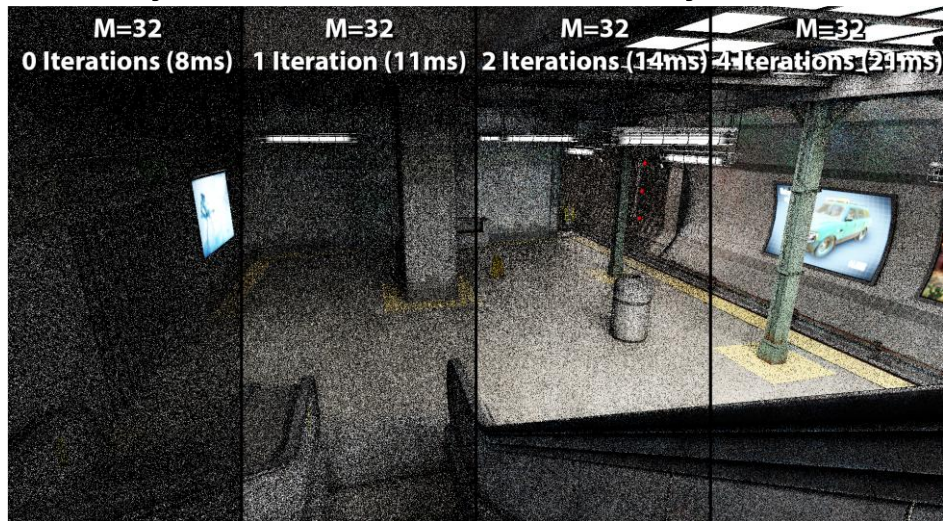
ReSTIR

- ▶ Resample contributions
 - Spatially from randomly selected nearby pixels
 - Temporally from previous frame



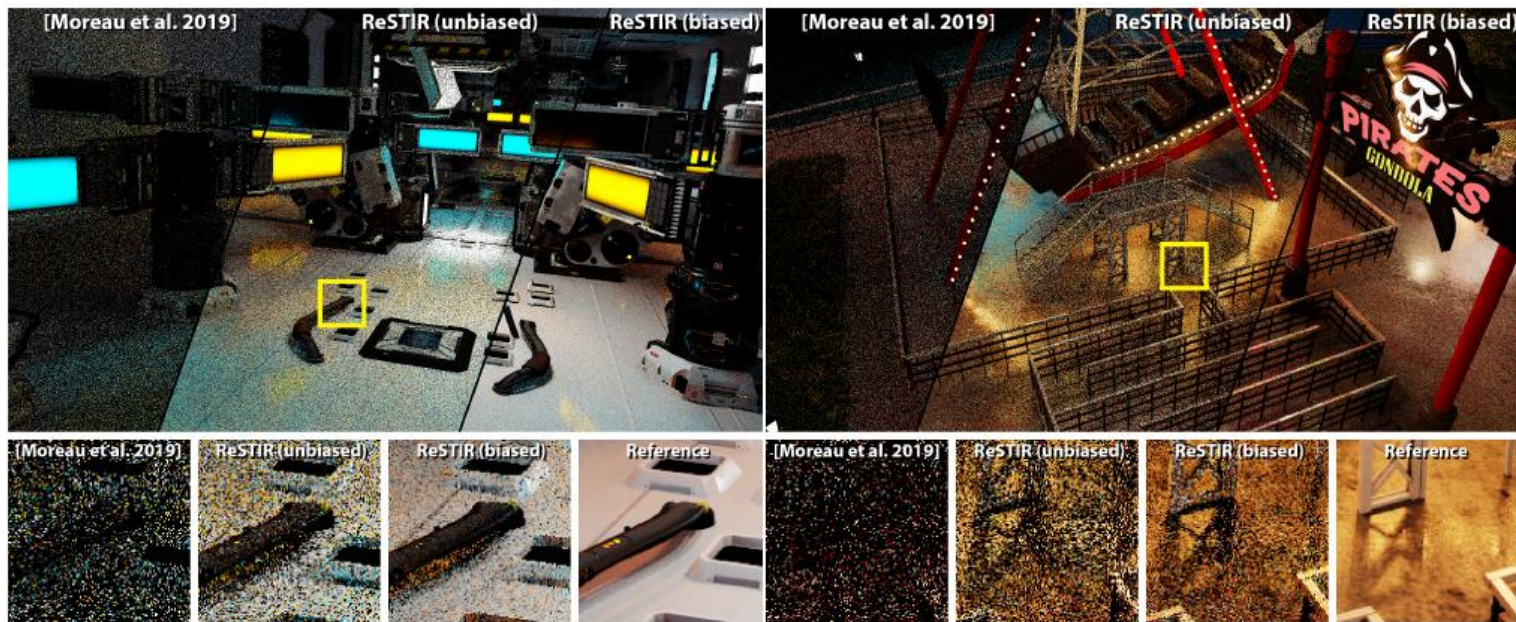
ReSTIR

- ▶ Resample contributions
 - Temporally biased if visibility not considered



ReSTIR

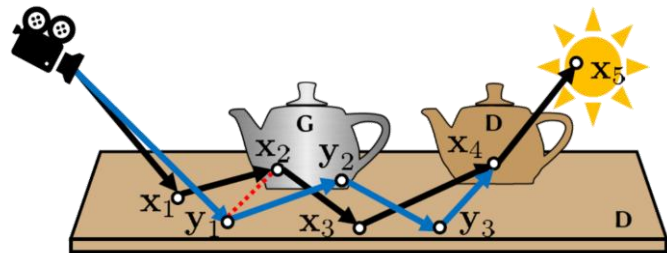
► Direct Lighting



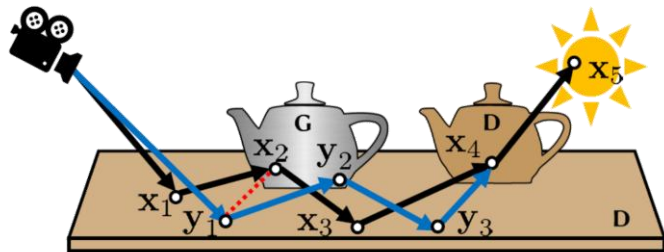
ReSTIR

► Path Tracing

- Needs to map nearby samples spatially and temporally
 - Only need to map needed vertices from screen
 - Keep rest of the path



ReSTIR



► Path Tracing

– Need to adjust for change in density

- Multiply by Jacobian

– Can directly connect

– Can retrace (glossy)

$$\left| \frac{\partial \omega_i^y}{\partial \omega_i^x} \right| = \frac{\cos \theta_2^y \|\mathbf{x}_{i+1} - \mathbf{x}_i\|^2}{\cos \theta_2^x \|\mathbf{x}_{i+1} - \mathbf{y}_i\|^2}$$

$$\left| \frac{\partial \omega_i^y}{\partial \omega_i^x} \right| = \left| \frac{\partial \omega_i^y}{\partial \tilde{u}_i^y} \right| \left| \frac{\partial \tilde{u}_i^y}{\partial \tilde{u}_i^x} \right| \left| \frac{\partial \tilde{u}_i^x}{\partial \omega_i^x} \right| = \frac{p_{\omega_i^x}(x_{i+1})}{p_{\omega_i^y}(y_{i+1})}$$

$$\left| \frac{\partial \tilde{u}_i^y}{\partial \tilde{u}_i^x} \right| = \left| \frac{\partial \tilde{u}_i^y}{\partial \omega_i^y} \right| \left| \frac{\partial \omega_i^y}{\partial \omega_i^x} \right| \left| \frac{\partial \omega_i^x}{\partial \tilde{u}_i^x} \right| = \frac{p_{\omega_i^y}(y_{i+1})}{p_{\omega_i^x}(x_{i+1})} \left| \frac{\partial \omega_i^y}{\partial \omega_i^x} \right|$$

ReSTIR

► Path Tracing



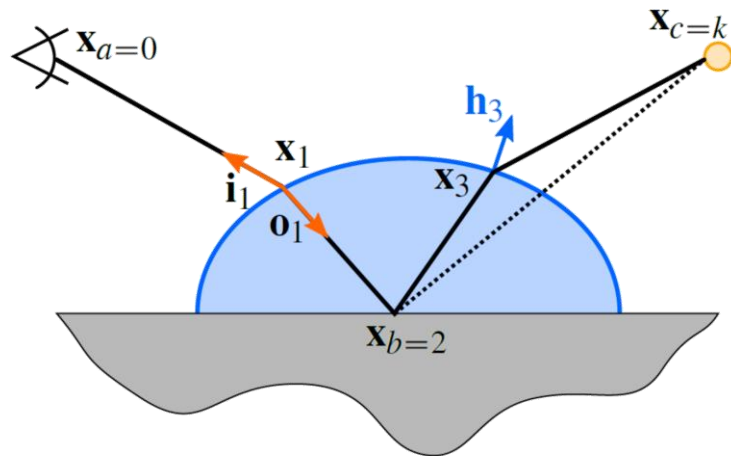
ReSTIR

► Path Tracing



Manifold Next Event Estimation

- ▶ Direct lighting through a specular surface
 - Refine an initial path by optimizing specular interaction positions



Manifold Next Event Estimation

- ▶ Direct lighting through a specular surface



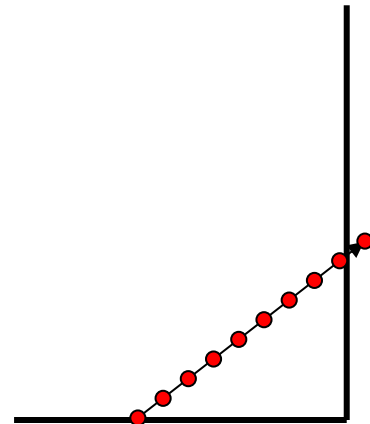
Screen Space Effects

- ▶ Use information in GBuffer to render effects
- ▶ Sample buffers to compute cheap approximation of expensive Monte Carlo integration



Screen Space Effects

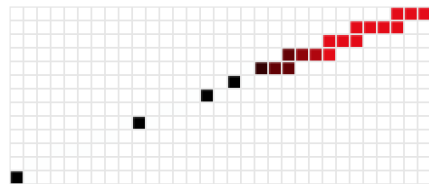
▶ Ray marching in screen space



Screen Space Effects

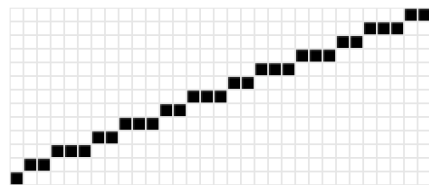
► Ray marching in screen space

- Step along ray and project
 - Issues with perspective projection



- Or calculate direction in screen space and step along ray

- More queries but more accurate



Screen Space Effects

- ▶ SSAO
- ▶ SSRT
- ▶ AA
- ▶ Motion Blur / DoF



Screen Space Effects

► Screen Space Ambient Occlusion

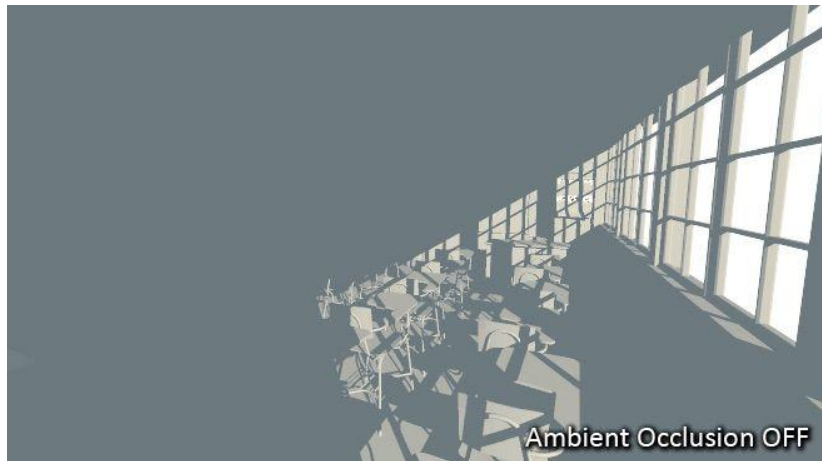
- Start with ambient occlusion
 - Simulates darkening in corners
 - Estimate ambient lighting

$$AO(x) = \frac{1}{\pi} \int_{\Omega} V'(x, \omega) \cos(\theta) d\omega$$

- Where $V'(x, \omega)$ is a binary visibility function
- Compute with Monte Carlo estimate

Screen Space Effects

- ▶ Screen Space Ambient Occlusion
 - If $V'(x, \omega)$ returns 1 when escaping the scene



Screen Space Effects

- ▶ Screen Space Ambient Occlusion
 - If $V'(x, \omega)$ returns 1 when no intersections within finite distance



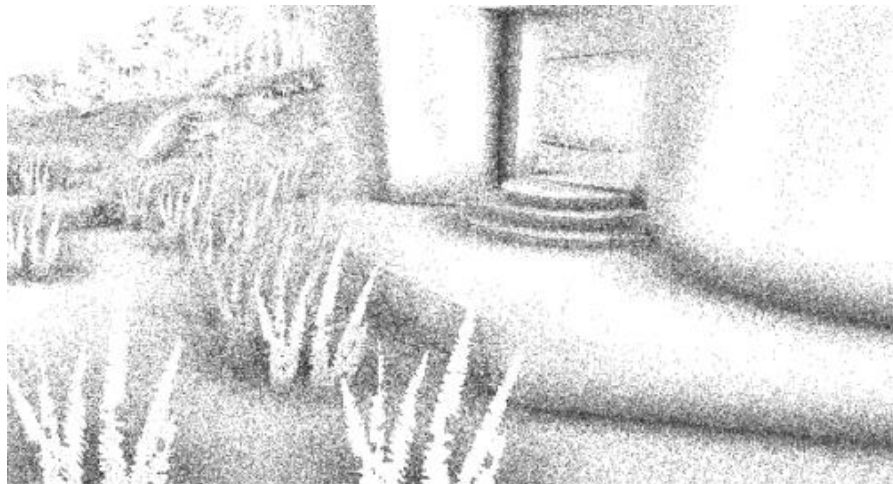
Screen Space Effects

- ▶ Screen Space Ambient Occlusion
 - Can estimate this in screen space
 - For each shading point
 - Sample pixels around the point
 - Estimate occlusion factor from position and normal from Gbuffer
 - Smooth noisy AO estimates



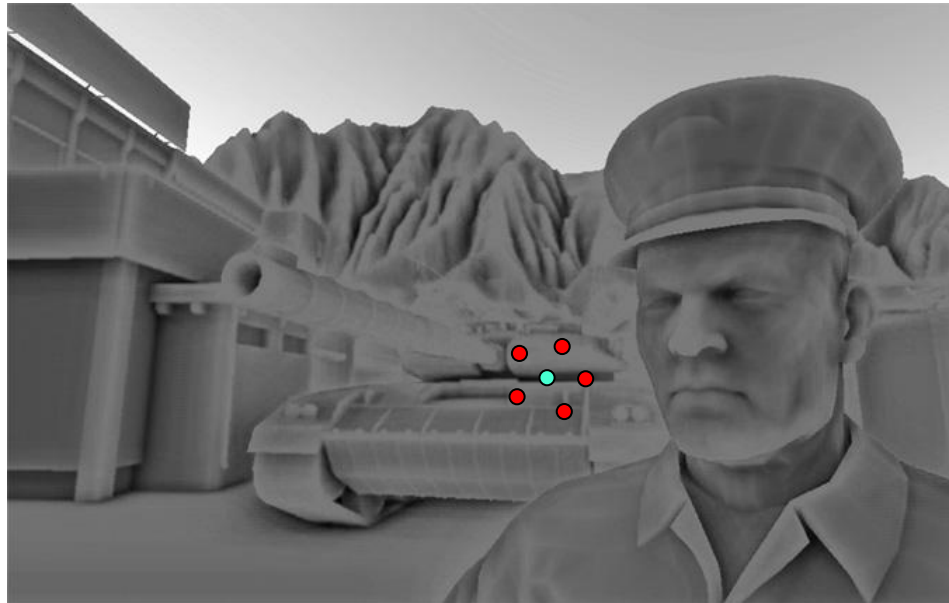
Screen Space Effects

▶ Screen Space Ambient Occlusion



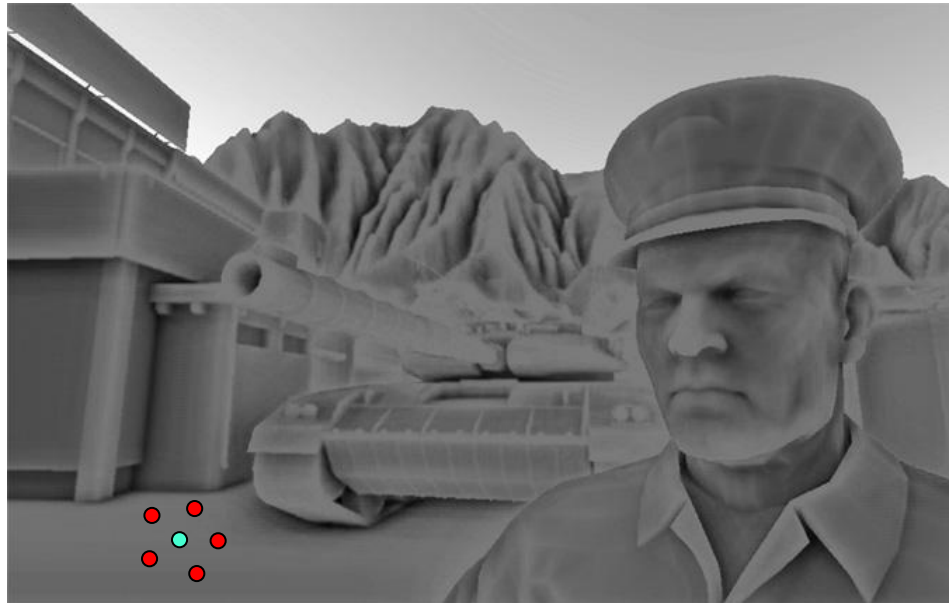
Screen Space Effects

- ▶ Screen Space Ambient Occlusion
 - Typical usage



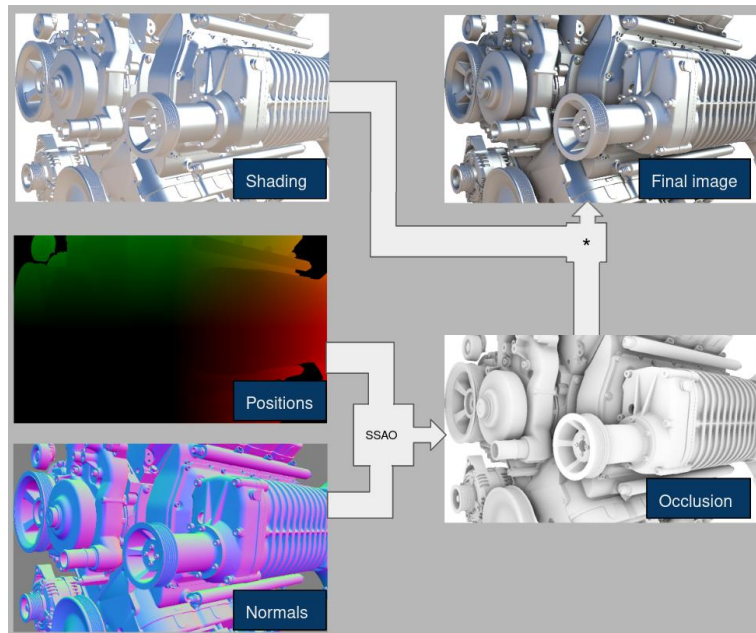
Screen Space Effects

- ▶ Screen Space Ambient Occlusion
 - Typical usage



Screen Space Effects

- ▶ Screen Space Ambient Occlusion
 - Typical usage



Screen Space Effects

- ▶ Screen Space Ambient Occlusion
 - Extensions:
 - HBAO (Horizon-Based Ambient Occlusion)



Screen Space Effects

- ▶ Screen Space Ray Tracing
 - Compute reflected ray direction and ray march



Screen Space Effects

- ▶ Screen Space Ray Tracing
 - Often used for water



Open Beta

Screen Space Effects

- ▶ Screen Space Ray Tracing Water
 - How to handle reflections outside frustum



Screen Space Effects

- ▶ Screen Space Ray Tracing Water
 - How to handle reflections outside frustum



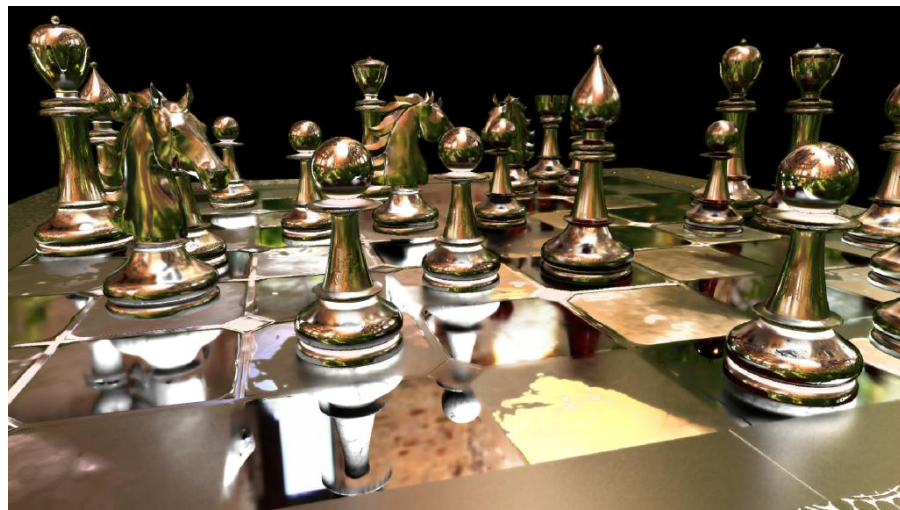
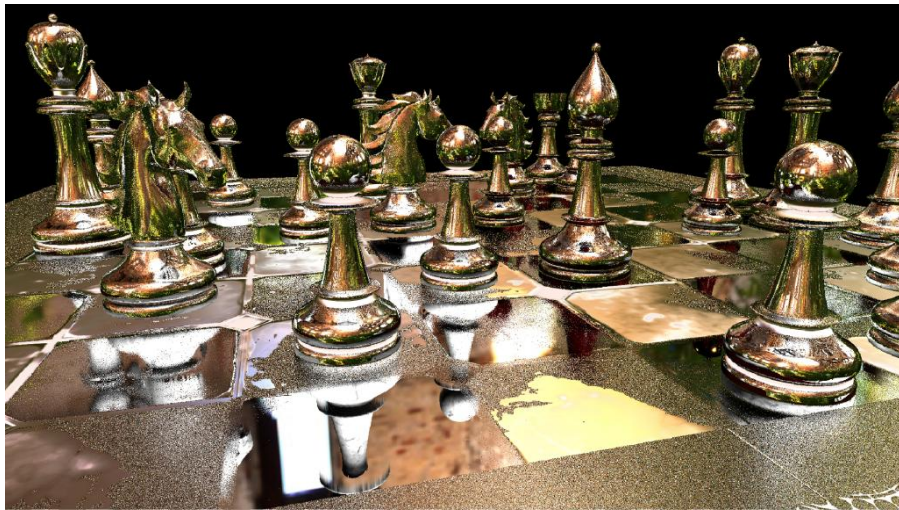
Screen Space Effects

- ▶ Screen Space Ray Tracing Water
 - How to handle gaps



Screen Space Effects

- ▶ Screen Space Ray Tracing
 - Can sample glossy BSDF (noisy)



Screen Space Effects

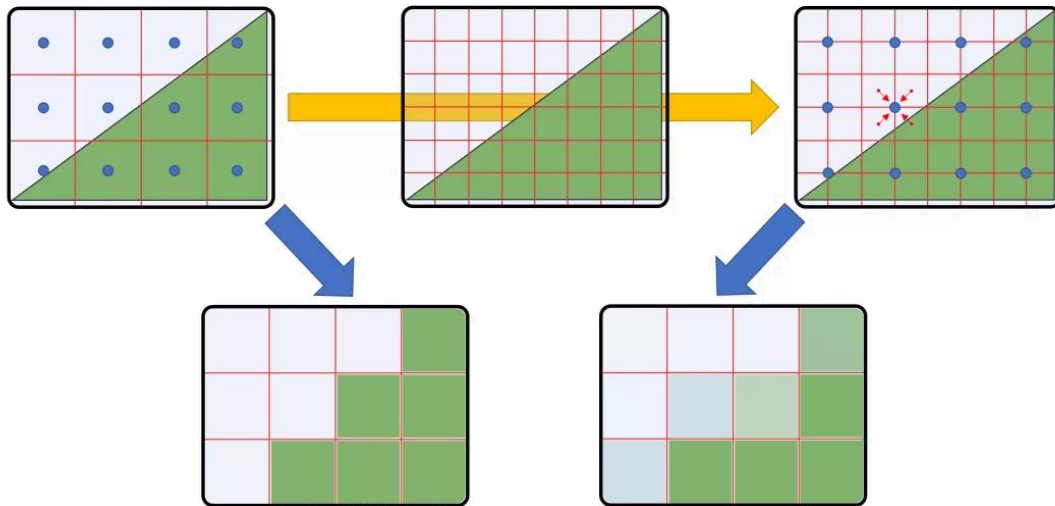
- ▶ Screen Space Ray Tracing
 - Or compute GI (fallback for HW raytracing GI)



Screen Space Effects

► Antialiasing

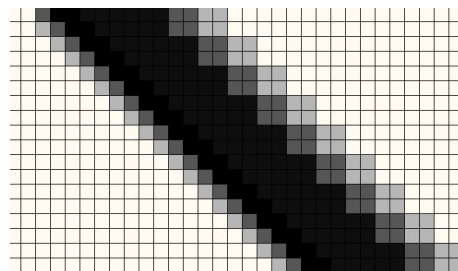
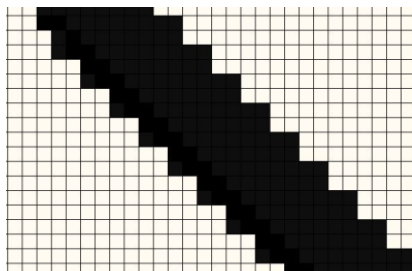
- Fix artifacts due to undersampling



Screen Space Effects

► Antialiasing

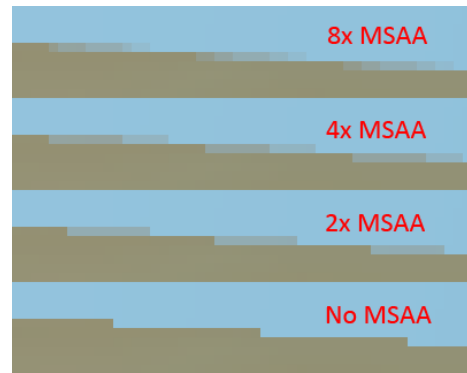
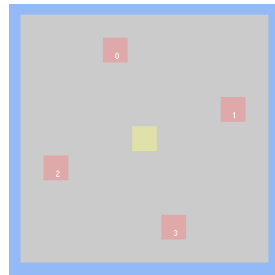
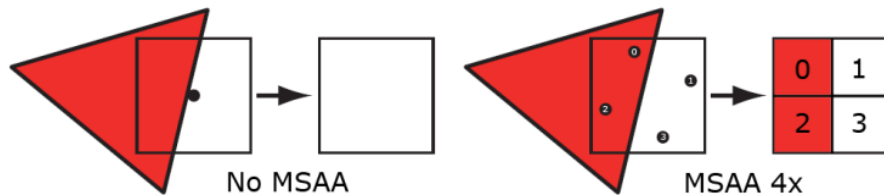
- SSAA (Super Sampling Anti Aliasing)
- Renders the image at a higher resolution and downsamples to screen size
 - Expensive



Screen Space Effects

► Antialiasing

- MSAA (Multi Sampling Anti Aliasing)
- Add more samples at geometric edges
- Average colours (Resolve)



Screen Space Effects

▶ Antialiasing

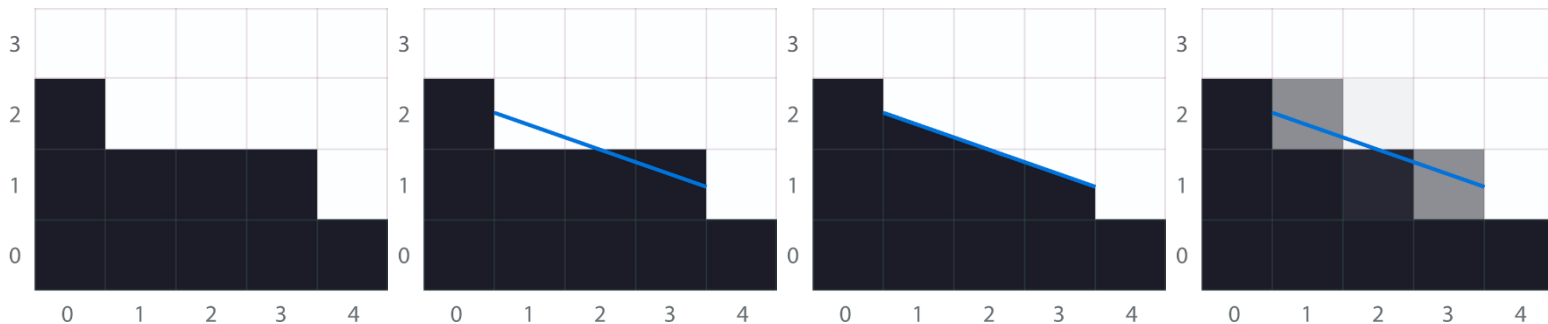
- SMAA (Subpixel Morphological Anti Aliasing)
- Try to fit smooth geometry to edges
 - Based on edge detection
- Analytically estimate image



Screen Space Effects

► Antialiasing

– SMAA (Subpixel Morphological Anti Aliasing)



Screen Space Effects

► Antialiasing

- SMAA (Subpixel Morphological Anti Aliasing)



Screen Space Effects

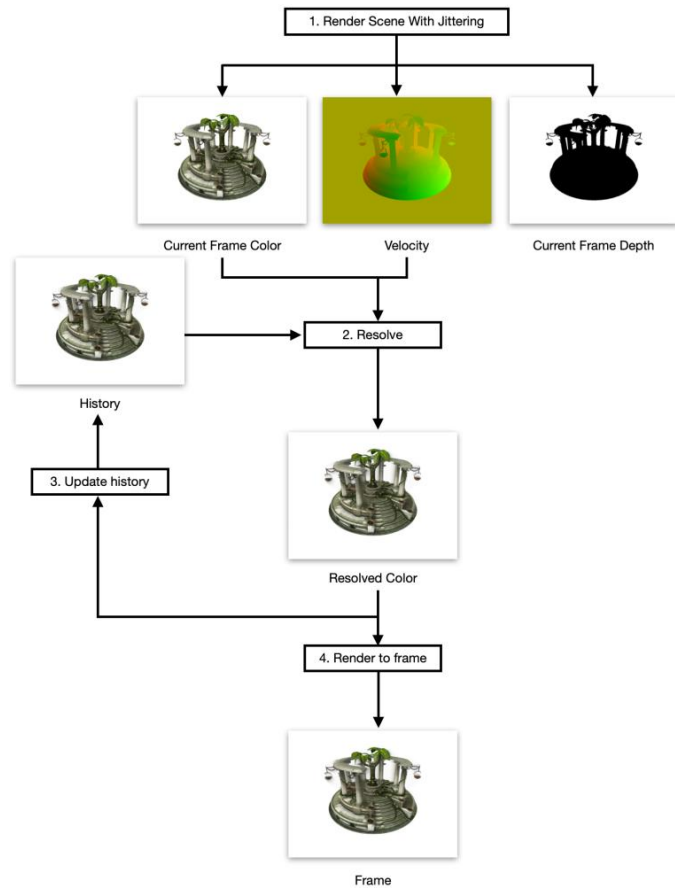
▶ Antialiasing

- TAA (Temporal Anti Aliasing)
- Uses reprojection from previous frame (i.e. motion vectors) and averages
 - Cheap and powerful
- Often jitter view matrix to provide variation



Screen Space Effects

- ▶ Antialiasing
 - TAA (Temporal Anti Aliasing)



Screen Space Effects

▶ Motion Blur

- Simulates integration over time
- Use temporal reprojection to compute motion vectors
- Sample pixels along motion vectors to compute blur



Screen Space Effects

► Motion Blur

- Simulates integration over time

$$L_o(x, \omega_o) = \int_{t_0}^{t_1} \left[L_e(x, \omega_o, t) + \int_{\Omega} f_r(x, \omega_i, \omega_o) L_i(x, \omega_i, t) (\omega_i \cdot n) d\omega_i \right] dt$$



Screen Space Effects

► Motion Blur



Screen Space Effects

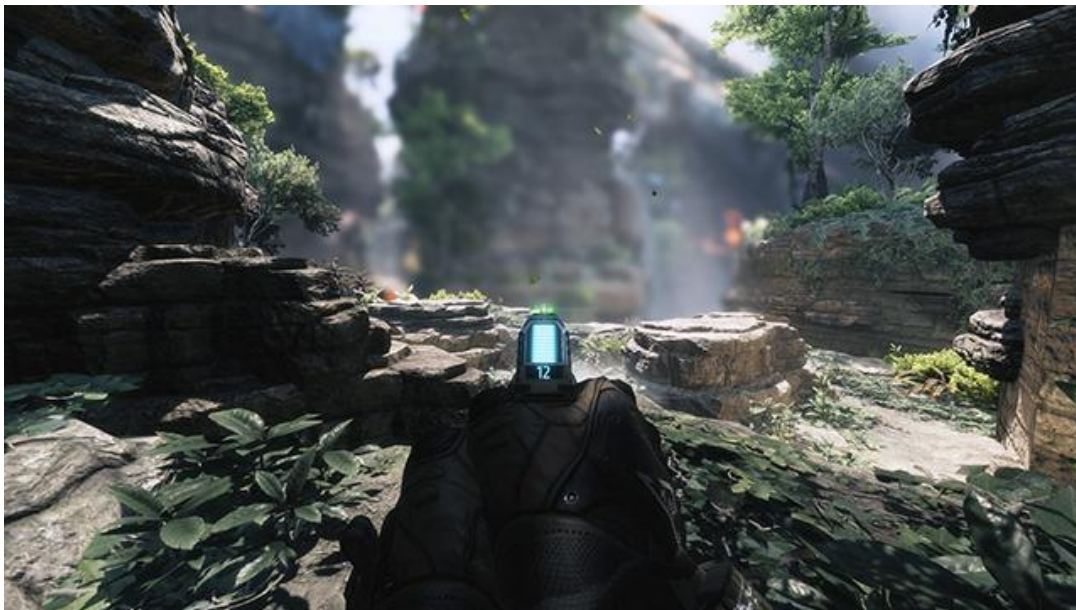
▶ Depth of Field

- Simulates a camera
- Use depth from depth buffer to select filter radius
 - Focal distance from centre pixel
- Usually use Gaussian blur



Screen Space Effects

► Depth of Field



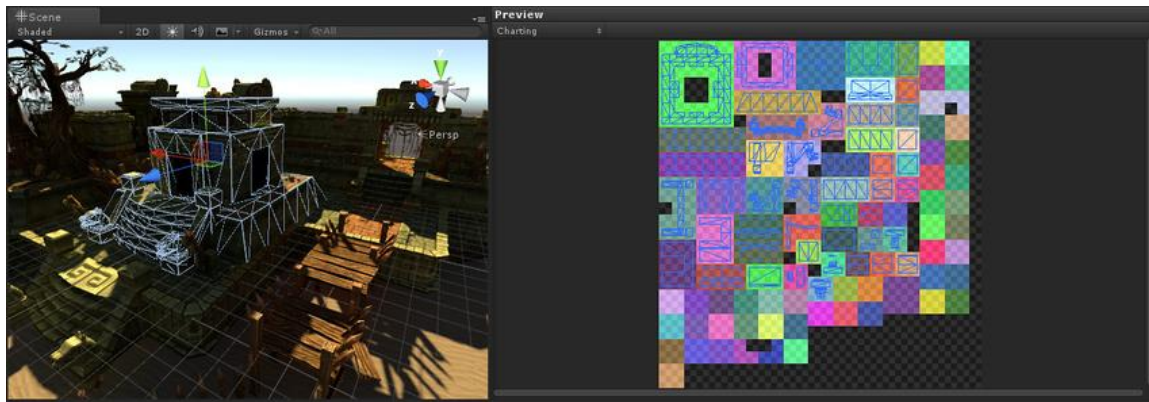
World Space Effects

- ▶ Lightmapping
- ▶ Voxel Based Global Illumination
- ▶ Precomputed Radiance Transfer
- ▶ Virtual Texturing
- ▶ Nanite



Lightmapping

- ▶ Store textures containing lighting
 - Often baked on level creation
 - Path Tracing from surfaces / IC / PM etc



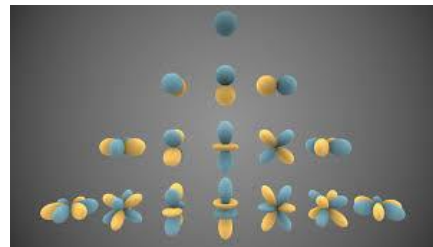
Voxel Based Global Illumination

- ▶ Map geometry into voxels
- ▶ Trace through voxels to compute lighting



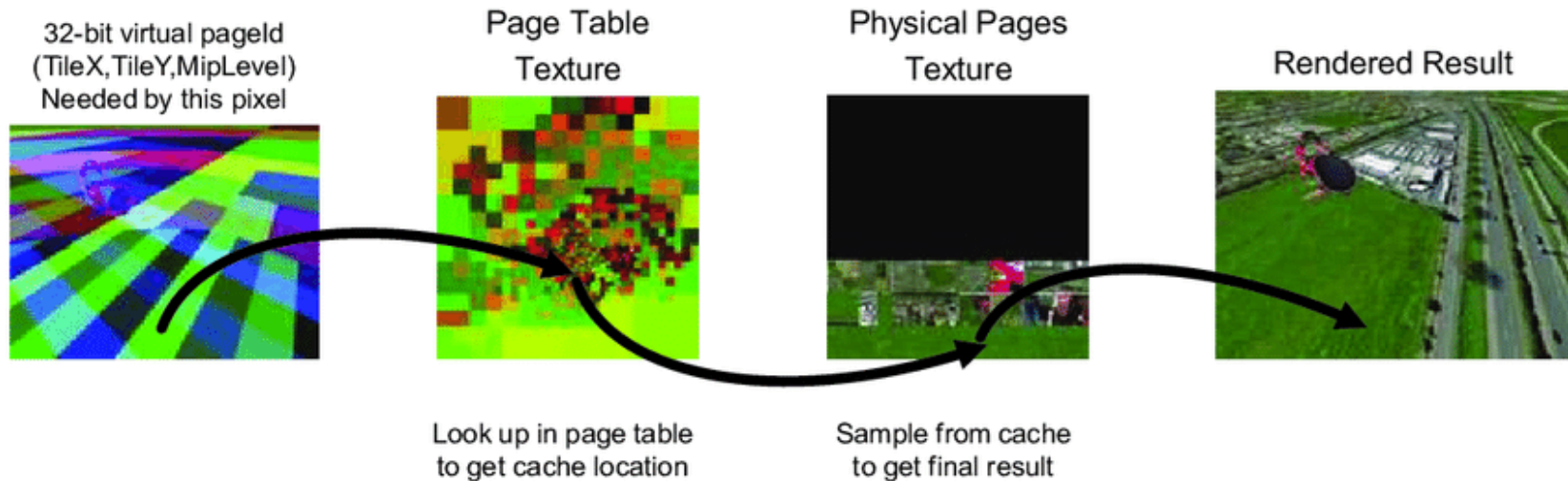
Precomputed Radiance Transfer

- ▶ Store directional lighting response
 - Usually from environment maps
 - Uses Spherical Harmonics



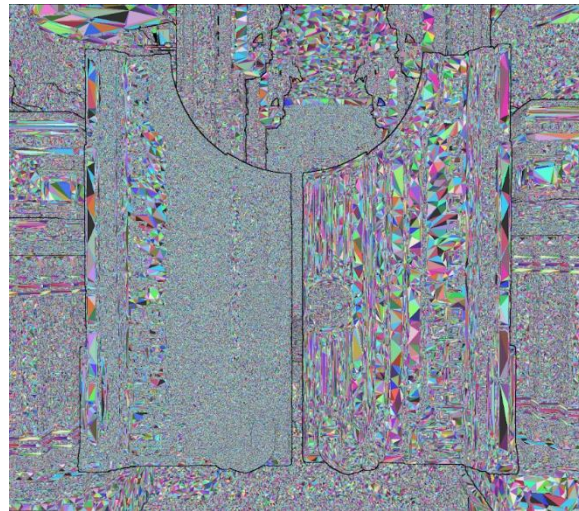
Virtual Texturing

- ▶ Only store some textures on GPU
- ▶ Stream in textures as needed



Nanite

- ▶ Only store some geometry on GPU
- ▶ Stream in meshes as needed

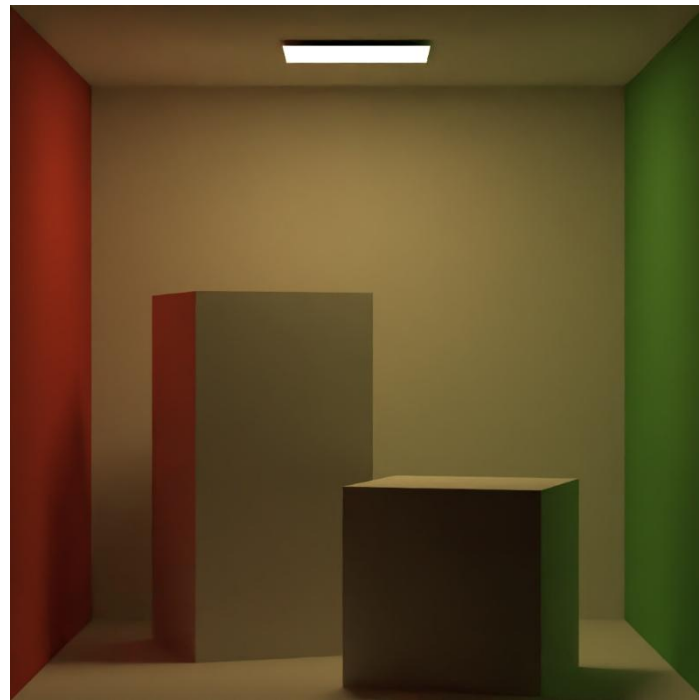
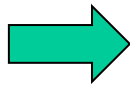
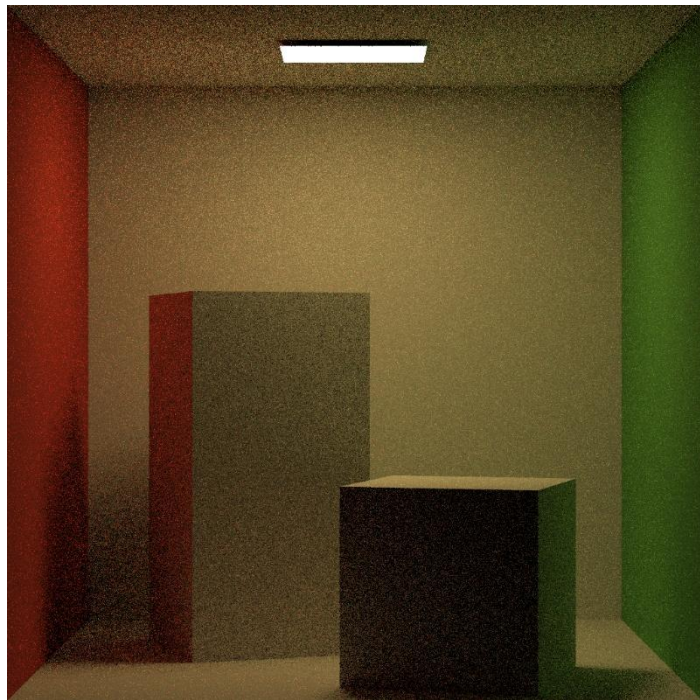


Denoising

- ▶ Rendered images are noisy(!)
- ▶ Want to decrease noise
 - Better sampling
 - Remove noise on image with denoisers
 - Guess what the image looks like without noise
 - Want to keep details while reducing noise



Denoising



Denoising

► General Aim

- Denoise noisy image $I(x) \in \mathbb{R}^3$

$$I_{filtered}(x) = \sum_{y \in \mathcal{W}(x)} w(x, y) I(y) \quad \text{with} \quad w(x, y) \propto \text{similarity}(x, y)$$

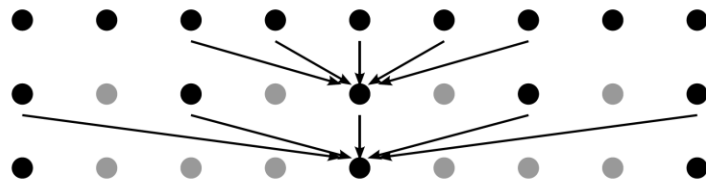
- Where $\mathcal{W}(x)$ is a “window” of pixels to filter



Denoising

▶ Two broad approaches:

- Classical
 - Filters
- Deep Learning based
 - Learns to denoise
 - e.g. DLSS

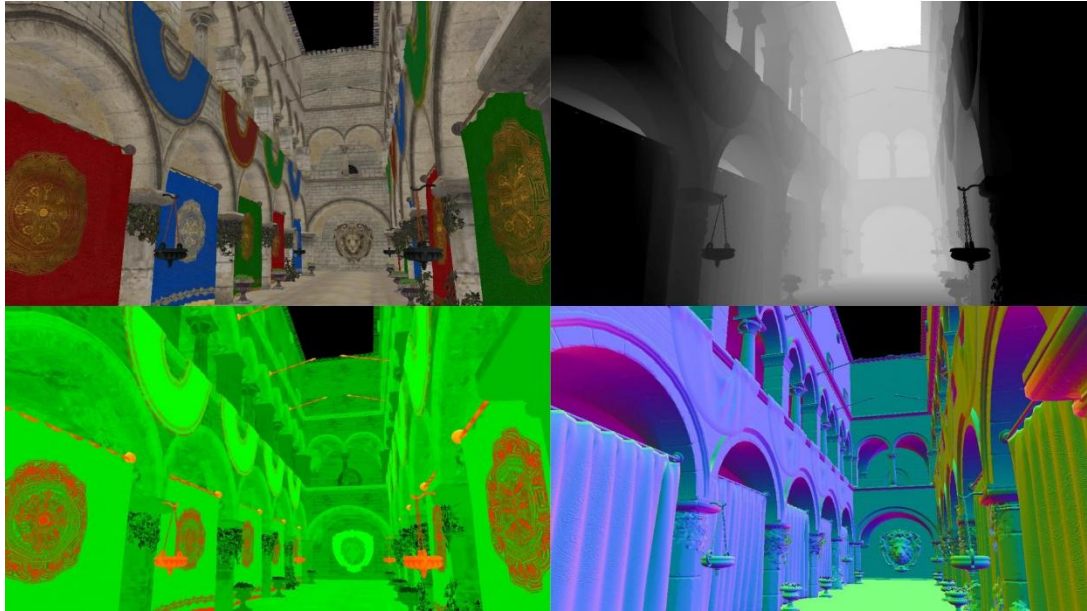


Denoising

- ▶ Arbitrary Output Variables (AOVs)
 - Buffers from the renderer used to help denoising (e.g. GBuffer features)
 - Often normals, depth, albedo, motion vectors, emission, secondary lighting effects (shadows, indirect lighting etc)
 - Will define these as $f_i(x) \in \mathbb{R}^d$

Denoising

► Arbitrary Output Variables (AOVs)



Denoising



Classical

- Incident radiance often changes slowly
- Can filter / average values to remove noise
- Want to base filter on image space features
 - Not over edges
 - Only spatially similar samples
 - Similar motivation to Irradiance Caching



Denoising



Classical

- Bilateral / Cross Bilateral Filter
- Weighted Gaussian Blur of image
- Weights based on
 - Spatial differences σ_x^2
 - Difference of features $\sigma_{f_i}^2$
 - Each can have custom deviation



Denoising

► Classical

- Bilateral / Cross Bilateral Filter

$$I_{filtered}(x) = \frac{1}{W(x)} \sum_{y \in \mathcal{W}(x)} w(x, y) I(y)$$

- Where (for one feature i)

$$w(x, y) = \exp \left(\frac{-\|x - y\|^2}{2\sigma_x^2} + \frac{-\|f_i(x) - f_i(y)\|^2}{2\sigma_{f_i}^2} \right)$$

- And

$$W(x) = \sum_{y \in \mathcal{W}(x)} w(x, y)$$

Denoising



Classical

- Non Local Means
- Compare patches around pixels and average
- Patches can be spatially distant
 - Expensive
- Exploits self similarity in images (preserves texture well)



Denoising



Classical

- Non Local Means

$$I(x) = \frac{1}{Z(x)} \sum_{y \in \mathcal{W}(x)} I(y) \cdot \exp \left(-\frac{\|\mathcal{W}(x) - \mathcal{W}(y)\|^2}{h^2} \right)$$

- Where $\mathcal{W}(x)$ is a patch, $Z(x)$ normalizes and h controls how similar patches need to be for denoising



Denoising



Classical

- À-Trous Wavelet Filter
 - Wavelets are a basis function
 - Edge avoiding
- Hierarchical filtering
 - Filters over larger region



Denoising



Classical

– À-Trous Wavelet Filter

$$I_{i+1}(p) = \frac{1}{K} \sum_{q \in \mathcal{W}_i(p)} h_i(q) \cdot w(p, q) \cdot I_i(q)$$

- With weight

$$w(p, q) = \exp \left(-\frac{\|I_p - I_q\|^2}{\sigma_{rt}^2} \right) \cdot \exp \left(-\frac{\|n_p - n_q\|^2}{\sigma_n^2} \right) \cdot \exp \left(-\frac{\|x_p - x_q\|^2}{\sigma_x^2} \right)$$



Denoising

► Classical

– À-Trous Wavelet Filter

- $h_i(q)$ is a wavelet
 - BSpline interpolating filter used in practice

$$h_0 = \left[\frac{1}{16}, \frac{1}{4}, \frac{3}{8}, \frac{1}{4}, \frac{1}{16} \right]$$

- Zeros inserted at higher levels
- $K = \sum_{q \in \mathcal{W}_i(p)} h_i(q) \cdot w(p, q)$ is a normalization term

Denoising

- ▶ Classical
 - À-Trous Wavelet Filter



Denoising

- ▶ Deep Learning based
 - Estimate denoised image using learned parameters instead of hand crafted values
 - Options:
 - Estimate filter kernel
 - Estimate denoised image directly



Denoising

► Deep Learning based

- Neural network estimates filter kernel

$$I_{filtered}(x) = \frac{1}{W(x)} \sum_{y \in \mathcal{W}(x)} w_{nn}(x, y) I(y), \quad W(x) = \sum_{y \in \mathcal{W}(x)} w_{nn}(x, y)$$

- Weight nearby pixels according to the kernel
 - Different kernel for every pixel
 - Kernel learnt from noisy image and features

Denoising

► Kernel Predicting

- Kernel-Predicting Convolutional Networks for Denoising Monte Carlo Renderings (Bako et al., SIGGRAPH 2017)



Ours



Input (32 spp)



Ours



Ref. (1K spp)

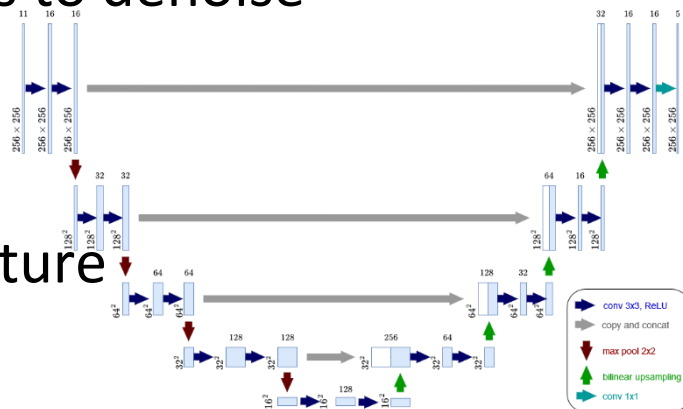


Denoising

► Estimate denoised image directly

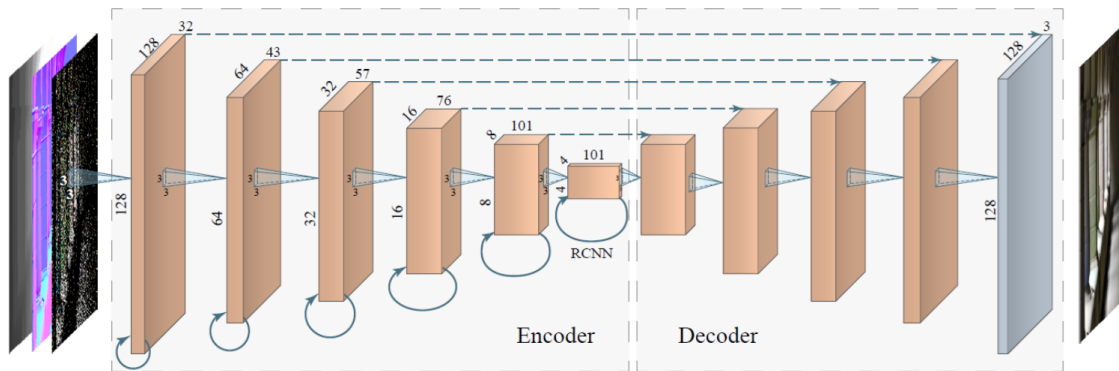
– Denoising Autoencoders

- Learns how to combine features to denoise
- Encode noisy image
- Decode denoised image
- Often based on a UNet architecture



Denoising

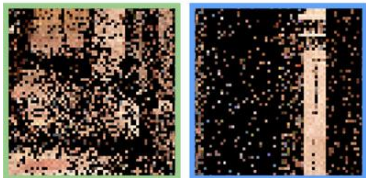
- ▶ Estimate denoised image directly
 - Interactive Reconstruction of Monte Carlo Image Sequences using a Recurrent Denoising Autoencoder (Chaitanya et al., SIGGRAPH 2017)



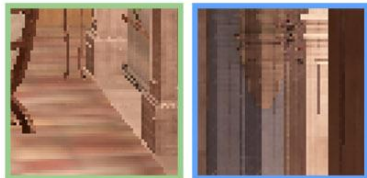
Denoising

► Estimate denoised image directly

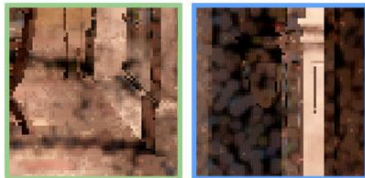
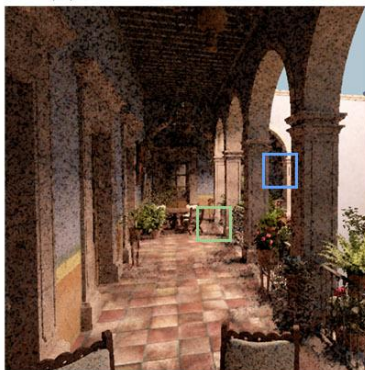
(a) 1spp noisy input



(b) Edge-avoiding wavelets



(c) SURE-based filter



(d) Recurrent autoencoder



(e) Reference



Denoising

- ▶ Estimate denoised image directly
 - [Open Image Denoise](#)



Denoising

- ▶ Estimate denoised image directly
 - [Open Image Denoise](#)
 - Used by film studios
 - Very simple to integrate (assignment)
 - Can use AOVs (normal, albedo) for guiding denoising
 - Use these in your assignment

Denoising

- ▶ Estimate denoised image directly
 - [Open Image Denoise](#)
 - Denoising autoencoder
 - Based on Unet
 - Convolutional Layers
 - Skip connections, RELU, L2 Loss



Denoising

- ▶ Estimate denoised image directly
 - [Open Image Denoise](#)
 - Need to create multiple temporary buffers
 - Film / SPP `colorBuffer`
 - Normals `normalBuffer`
 - Albedo `albedoBuffer`
 - Output `outputBuffer`

Denoising

► Estimate denoised image directly

— Open Image Denoise `#include <OpenImageDenoise/oidn.hpp>`

```
oidn::DeviceRef device = oidn::newDevice();
device.commit();
oidn::FilterRef filter = device.newFilter("RT");
filter.setSharedImage("color", colorBuffer, oidn::Format::Float3, width, height);
filter.setSharedImage("albedo", albedoBuffer, oidn::Format::Float3, width, height);
filter.setSharedImage("normal", normalBuffer, oidn::Format::Float3, width, height);
filter.setSharedImage("output", outputBuffer, oidn::Format::Float3, width, height);
filter.set("hdr", true);
filter.commit();
filter.execute();
```

Denoising

- ▶ Estimate denoised image directly
 - [Open Image Denoise](#)
 - Copy denoised pixels out of `outputBuffer`
 - Can tonemap the denoised output to screen
 - Or save to HDR file



DLSS / FSR

- ▶ Combines denoising with upsampling
 - Can upsample spatially
 - Can temporally (DLSS 4)
- ▶ Uses reprojection of previous frames + ML
- ▶ Trained from huge database of images



DLSS

▶ DLSS 3

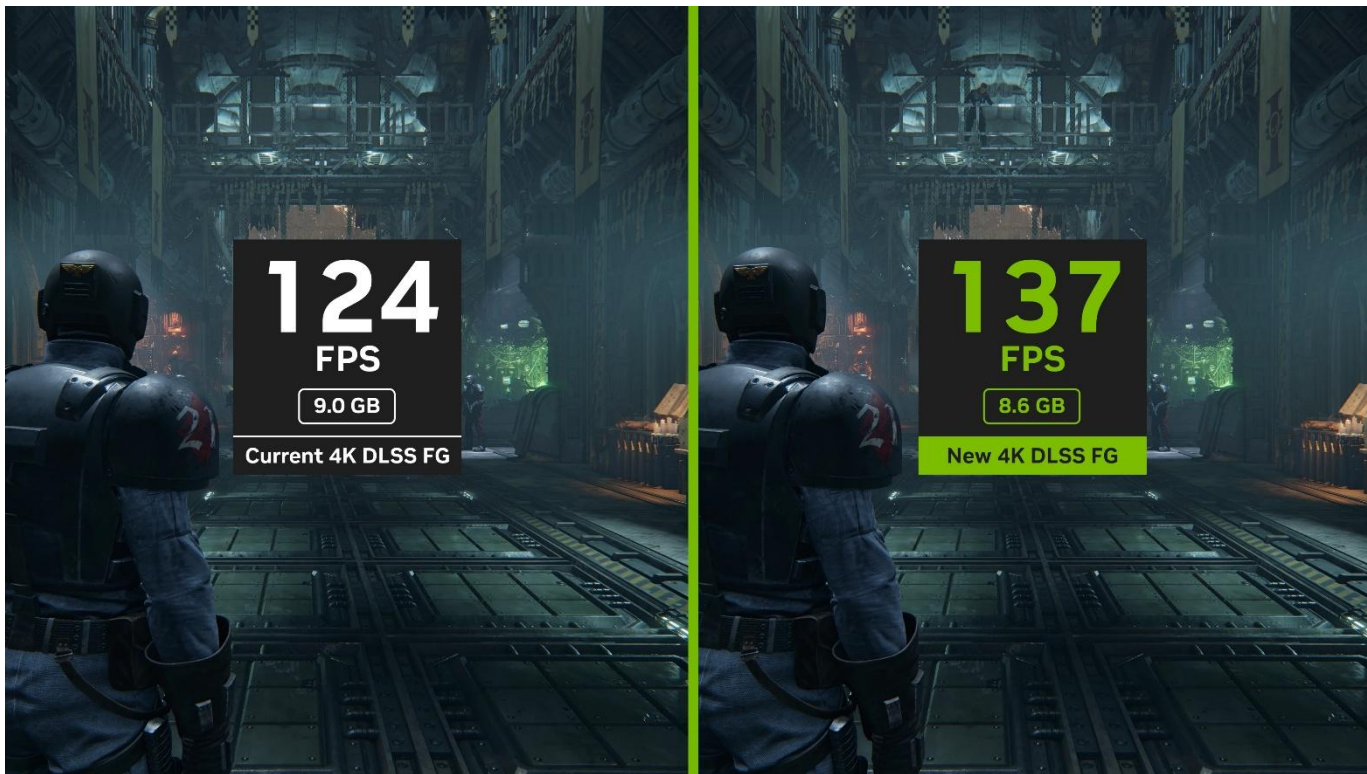
- Convolutional Neural Networks for upsampling

▶ DLSS 4

- Transformers
- Frame interpolation



DLSS



FSR

- ▶ Based on Machine Learning and optical flow
- ▶ Jitters frames to compute subpixel details



FSR



RDNA 4 PATH TRACING
WITH NEURAL SUPERSAMPLING AND DENOISING

AMD
together we advance gaming

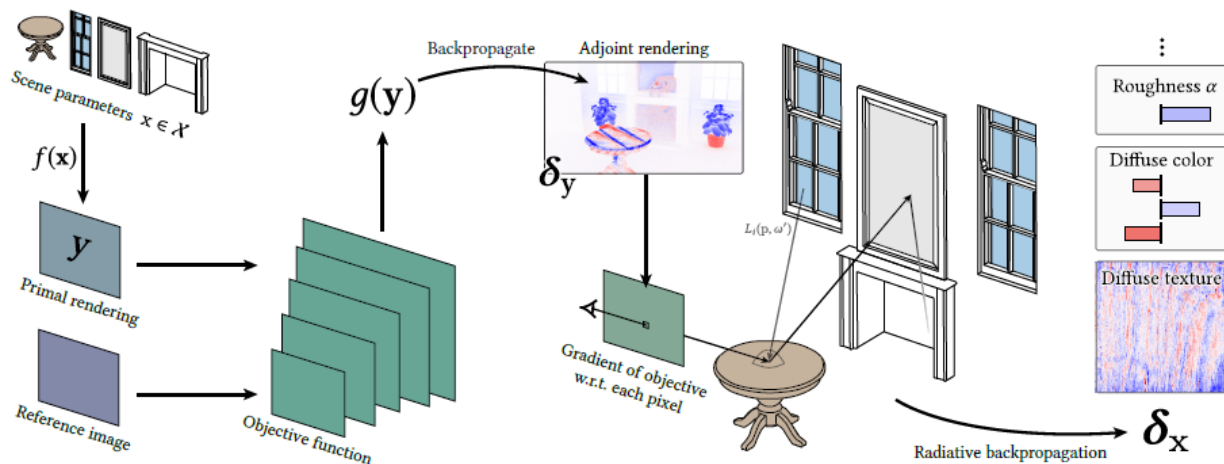
State-Of-The-Art Research

- ▶ Differentiable Rendering
- ▶ Accurate Material Models
- ▶ Machine Learning
- ▶ Quantum Rendering

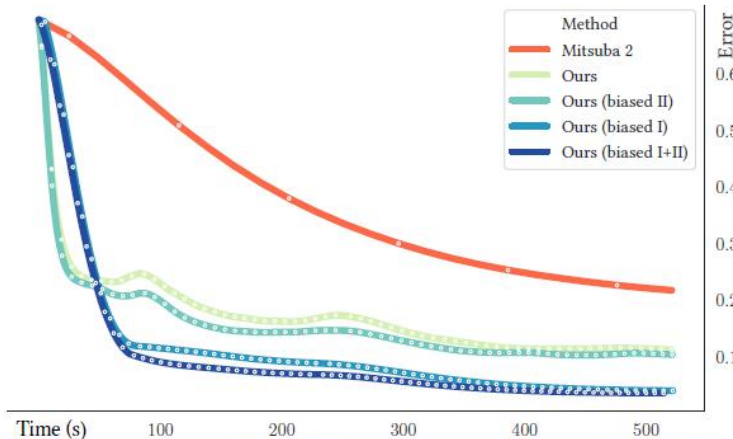


Differentiable Rendering

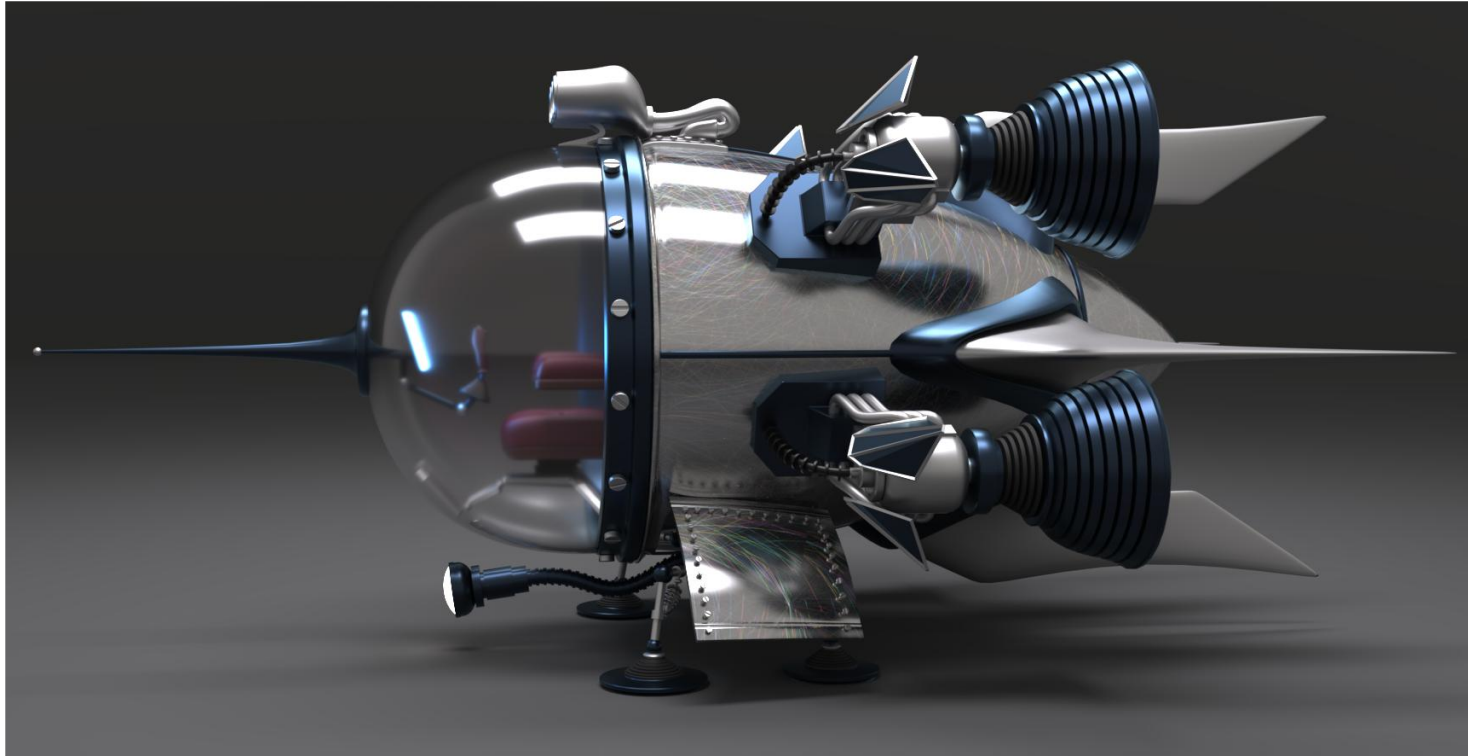
- ▶ Calculate derivatives of light paths
- ▶ Can be respect to parameters to optimize



Differentiable Rendering



Accurate Material Models



Machine Learning



Quantum Rendering



Implementation

- ▶ Add denoiser
- ▶ Assignment





Questions?

Computer Graphics

Core Mathematical Concepts



WARWICK

Dr. Tom Bashford-Rogers